Trabajo Práctico 8 — Resolución

Programación Lógica Rebatible (DeLP)

Inteligencia Artificial — UNS — 2026

Las fuentes van en notas al pie. **Cátedra**: diapositivas de A. J. García (2026). **AIMA**: Russell & Norvig, *Artificial Intelligence: A Modern Approach*, 3.ª ed. (2010). **PMG**: Poole, Mackworth & Goebel, *Computational Intelligence: A Logical Approach* (1998).

Ejercicio 1: Conceptos básicos

a)

Consigna

¿Qué es un programa lógico rebatible?

Un **programa lógico rebatible** (PLR) es un par (Π, Δ) (o el conjunto $\Pi \cup \Delta$) formado por:[1]

- Π: un conjunto de **hechos** y **reglas estrictas**.
  - *Hecho*: un literal fijo (sin variables).
  - *Regla estricta* $L_0 \leftarrow L_1, \ldots, L_n$: información **segura**.
- Δ: un conjunto de **reglas rebatibles** $L_0 \prec L_1, \ldots, L_n$: información **tentativa**.

Un **literal** es un átomo $p(T_1, \ldots, T_n)$ o un átomo negado $\sim p(T_1, \ldots, T_n)$, donde "~" es la **negación fuerte**.

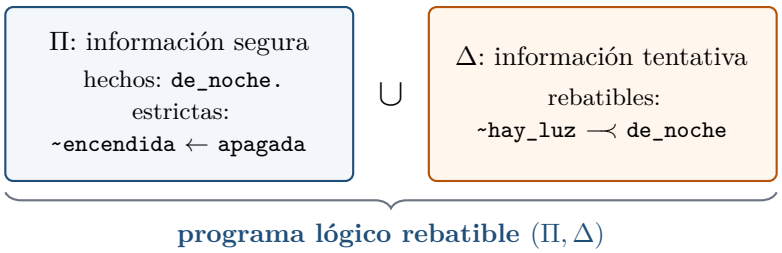


Figura 1: Componentes de un PLR (con reglas del inciso d como ejemplo).

EXTRAS

- DeLP provee un **lenguaje de representación de conocimiento** que permite representar información **tentativa** y **potencialmente en conflicto**. Ante una consulta, su mecanismo de inferencia construye y evalúa **todos** los argumentos relacionados con ella (a favor y en contra).[2]
- **Variables**: las mayúsculas en los literales denotan variables; una regla con variables es un **esquema de reglas**. Los hechos, en cambio, son los encargados de **propagar las instancias** en las derivaciones de las conclusiones.[3]
- La base es la tesis doctoral de A. J. García, *Programación Lógica Rebatible* (UNS, 2000), cap. 2, y el artículo de García y Simari, *Defeasible logic programming: An argumentative approach* (TPLP, 2004).[4]

[1] Cátedra, Episodio X, parte 2, «Reglas rebatibles (defeasible rules)» y «Hechos (facts)».
[2] Cátedra, Episodio X, parte 2, «Defeasible Logic Programing (DeLP)».
[3] Cátedra, Episodio X, parte 2, «Reglas rebatibles (defeasible rules)» y «Hechos (facts)».
[4] Cátedra, Episodio X, parte 2, «Defeasible Logic Programing (DeLP)».

b)

Consigna

¿Cuál es la utilidad de contar con dos tipos de reglas: estrictas y rebatibles? ¿Qué tipo de información permite modelar cada una de estas reglas? ¿Cómo se distinguen estas reglas en el lenguaje?

Qué modela cada una.[5]

- **Estricta**: información **segura** (no rebatible). La conexión entre el antecedente y la conclusión **se cumple siempre**: no puede haber excepciones. Ej.: `~abierto` ← `cerrado`.
- **Rebatible**: información **tentativa**. La conexión entre el antecedente (las razones) y la conclusión se cumple en muchos casos, pero **puede haber excepciones**. Ej.: `~cruzar` —≺ `semaforo(rojo)`.

Utilidad de tener las dos.

- Con las rebatibles se pueden expresar **reglas generales** que se cumplen en la mayoría de los casos y, además, reglas que dicen **en qué circunstancias hay excepciones**; también **posturas u opiniones diferentes** que pueden venir de distintas fuentes.[6]
- Con las estrictas se representa lo que no admite excepciones. Si hay excepciones, no es estricta y debe representarse como rebatible; la decisión depende del **dominio de aplicación** del SBC y es responsabilidad de quien codifica el conocimiento.[7]

Cómo se distinguen. Sintácticamente, **solo por la flecha** (aunque la diferencia semántica es enorme):[8]

Regla	Notación	Información que modela	Ejemplo
Estricta	$L_0 \leftarrow L_1, \ldots, L_n$	segura: se cumple siempre, sin excepciones	`~abierto` ← `cerrado`
Rebatible	$L_0 \prec\!\!- L_1, \ldots, L_n$	tentativa: se cumple en muchos casos, puede tener excepciones	`~cruzar` —≺ `semaforo(rojo)`

EXTRAS

- **Lectura de una regla rebatible** `conclusión` —≺ `antecedente`: "si hay razones para creer en el antecedente, entonces hay una razón (rebatible) para creer en la conclusión".[9]
- **Cadena de excepciones**: cada regla puede ser excepción de la anterior. Según los hechos que perciba el agente, pueden derivar conclusiones las tres, solo dos, solo una o ninguna:[10]

```
~cruzar —≺ semaforo(rojo).                          regla general
cruzar —≺ semaforo(rojo), emergencia.               excepción a la primera
~cruzar —≺ semaforo(rojo), emergencia, hay_peligro. excepción a la segunda
```

[5] Cátedra, Episodio X, parte 2, «Reglas rebatibles (defeasible rules)» y «Reglas estrictas (strict rules)».
[6] Cátedra, Episodio X, parte 2, «Defeasible Logic Programing (DeLP)».
[7] Cátedra, Episodio X, parte 2, «Reglas estrictas (strict rules)».
[8] Cátedra, Episodio X, parte 2, «Reglas estrictas (strict rules)».
[9] Cátedra, Episodio X, parte 2, «Reglas rebatibles (defeasible rules)».
[10] Cátedra, Episodio X, parte 2, «Reglas rebatibles (defeasible rules)».

c)

Consigna

¿Qué ventajas tiene usar DeLP como mecanismo de inferencia y/o como herramienta de representación de conocimiento frente a otros enfoques, como el de la lógica clásica o frente a Prolog?

Ventajas de DeLP.[11]

- *Representación*: permite representar información **tentativa** y información **potencialmente en conflicto**.
- *Inferencia*:
  - las conclusiones están **sustentadas con argumentos**;
  - las respuestas se basan en un **análisis exhaustivo** de todos los argumentos relacionados (a favor y en contra);
  - el criterio de preferencia entre argumentos se puede **adaptar al dominio** de aplicación;
  - se puede dar una **explicación detallada** de las respuestas.

Frente a Prolog.

- Prolog estándar **no tiene negación fuerte**, y tampoco un mecanismo para **manejar información contradictoria**: si hay derivaciones para `comprar_acciones(acme)` y para `no_comprar_acciones(acme)`, responde `true` a las dos.[12]
- Solo tiene **negación por falla** (`\+`), que no puede ir en la cabeza de una regla ni en un hecho.[13]
- DeLP agrega la negación fuerte (~) y las reglas rebatibles, y ante conclusiones contradictorias **decide cuál prevalece** comparando los argumentos a favor y en contra.[14]

Frente a la lógica clásica.

- La lógica clásica es **monotónica**:[15] "*the set of entailed sentences can only increase as information is added to the knowledge base*". Lo que se concluyó no se retracta al aparecer una excepción.
- Para no inferir conclusiones contradictorias con inferencia monotónica (como Modus Ponens) hay que **mantener la base libre de contradicciones**. La otra opción es **admitir contradicciones potenciales** en la base y tener una inferencia (no monotónica) que **decida** ante una contradicción.[16] DeLP sigue esta segunda opción.

	Lógica clásica	**Prolog**	**DeLP**
Negación	—	solo por falla (`\+`)	fuerte (~)
Información tentativa	no (monotónica)	no	sí (reglas rebatibles)
Contradicciones	base libre de contradicciones	concluye las dos	decide cuál prevalece

EXTRAS

[11] Cátedra, Episodio X, parte 2, «Defeasible Logic Programming». Mismo resumen en Episodio X (tercera parte), «Comentarios finales».
[12] Cátedra, Episodio VIII, «Extender el mecanismo de inferencia».
[13] Cátedra, Episodio VIII, «Negación por falla en Prolog».
[14] Cátedra, Episodio X, parte 2, «Defeasible Logic Programing (DeLP)».
[15] AIMA, §7.5, p. 251.
[16] Cátedra, Episodio IX (valores de certeza), «Bases de creencias inconsistentes: dos grandes opciones».

- **No monotonía en AIMA**: razonar por defecto, sacando conclusiones "*in the absence of any reason to doubt it*" y retractándolas si aparece nueva evidencia, es habitual en el sentido común. Ese razonamiento "*is said to exhibit nonmonotonicity, because the set of beliefs does not grow monotonically over time as new evidence arrives*". Ejemplos de lógicas no monotónicas: *circumscription* y *default logic*.[17]
- **Un paso intermedio sobre Prolog**: agregarle el operador ~ y una inferencia "cauta" que responda `si` (deriva Q pero no su complemento), `no` (deriva el complemento pero no Q), `indeciso` (deriva ambos) o `desconozco` (ninguno). Ante una contradicción no decide: se abstiene.[18]

---

d)

Consigna

Escriba reglas estrictas o rebatibles, según considere más adecuado, para representar el siguiente conocimiento:

I. Si la linterna está apagada, entonces no está encendida.
II. Si la linterna está encendida, entonces no está apagada.
III. Si es de noche, generalmente no hay luz.
IV. Si es de noche y tengo una linterna, entonces puedo iluminar.
V. Si la linterna no tiene batería, entonces no ilumina.
VI. Si es de noche y hay luna llena, entonces hay luz.

Supuesto

"Puedo iluminar" (iv) y "no ilumina" (v) se representan con el mismo literal `ilumina` (y su complemento), y "hay luz" (iii y vi) con `hay_luz`; si no, las reglas no entrarían en conflicto.

Criterio: si la regla puede tener excepciones, es rebatible; si se cumple siempre, estricta.[19]

	Regla	Tipo	Por qué
i	`~encendida` ← `apagada`	estricta	por definición: apagada y encendida son opuestos, sin excepciones
ii	`~apagada` ← `encendida`	estricta	ídem
iii	`~hay_luz` —≺ `de_noche`	rebatible	"generalmente": tiene excepciones (vi)
iv	`ilumina` —≺ `de_noche, tengo_linterna`	rebatible	tiene excepciones: v
v	`~ilumina` ← `~tiene_bateria`	estricta	sin batería, la linterna no ilumina nunca
vi	`hay_luz` —≺ `de_noche, luna_llena`	rebatible	es una excepción a iii, y a su vez puede tener excepciones (p. ej. noche nublada)

- iv no dice "generalmente", pero v es una excepción: tener linterna de noche no basta si no tiene batería. Si iv fuera estricta, con `de_noche`, `tengo_linterna` y `~tiene_bateria` en Π se derivarían `ilumina` y `~ilumina` solo con información segura.
- iii y vi entran en conflicto (`~hay_luz` vs. `hay_luz`) en una noche de luna llena: las dos son razones tentativas y será la comparación de argumentos la que decida.

[17] AIMA, §12.6.1, p. 458.
[18] Cátedra, Episodio VIII, «Extendiendo el mecanismo de inferencia».
[19] Cátedra, Episodio X, parte 2, «Reglas estrictas (strict rules)».

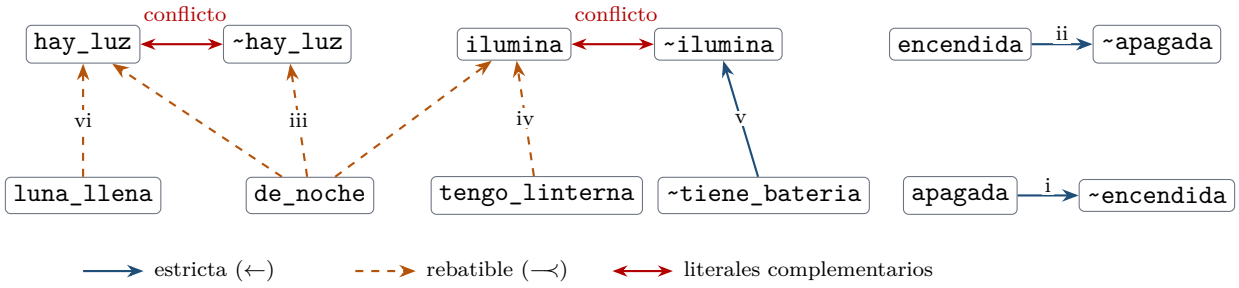


Figura 2: Las reglas del inciso d: de los antecedentes a la conclusión, y los conflictos entre conclusiones.

EXTRAS

- Con v estricta, si `~tiene_bateria` es un hecho, el conjunto {iv} junto con Π es contradictorio (deriva `ilumina` y `~ilumina`), así que no es un argumento para `ilumina`: no cumple la condición de consistencia.[20]

Ejercicio 2

Consigna

Considere el siguiente programa lógico rebatible $\mathcal{P}_2 = (\Pi_2, \Delta_2)$:

$$\Pi_2 = \left\{ \begin{array}{l} asustada(tina) \\ gallina(tina) \\ gallina(peluche) \\ \sim vuela(peluche) \\ ave(X) \leftarrow gallina(X) \end{array} \right\} \qquad \Delta_2 = \left\{ \begin{array}{l} vuela(X) \prec ave(X) \\ \sim vuela(X) \prec gallina(X) \\ vuela(X) \prec gallina(X), asustada(X) \\ anida_en_ramas_altas(X) \prec vuela(X) \end{array} \right\}$$

En lo que sigue, las reglas se nombran así:

E1:	`ave(X)` ← `gallina(X)`	R2:	`~vuela(X)` ≺ `gallina(X)`
R1:	`vuela(X)` ≺ `ave(X)`	R3:	`vuela(X)` ≺ `gallina(X), asustada(X)`
		R4:	`anida_en_ramas_altas(X)` ≺ `vuela(X)`

a)

Consigna

¿A qué se denomina derivación rebatible? Obtenga todas las derivaciones para los siguientes literales: vuela(tina), ~vuela(tina), asustada(tina), vuela(peluche) y ~vuela(peluche).

Derivación rebatible. Una **derivación rebatible** para un literal L a partir de un PLR (Π, Δ) es una secuencia finita de literales fijos $L_1, L_2, \ldots, L_n = L$ tal que, para cada L_i:[21]

- L_i es un **hecho** de Π, o
- existe en (Π, Δ) una **instancia de una regla** (estricta o rebatible) con cabeza L_i y cuerpo $B_1, \ldots, B_k$, donde todo literal del cuerpo ya está en $L_1, \ldots, L_{i-1}$.

Si solo usa hechos y reglas estrictas de Π, es una **derivación estricta**.

[20] Cátedra, Episodio X, parte 2, «Argumento».
[21] Cátedra, Episodio X, parte 2, «Derivaciones rebatibles (defeasible derivation)».

Qué se cuenta como "todas"

Cambiar el orden de literales independientes (p. ej. `asustada(tina)` antes o después de `gallina(tina)`) o agregar literales que no se usan da otras secuencias que también cumplen la definición. Se listan las derivaciones distintas en cuanto a **qué hechos y reglas usan**.

Literal		Secuencia	**Justificación de cada L_i**	**Tipo**
`vuela(tina)`	D1	`gallina(tina)`, `ave(tina)`, `vuela(tina)`	hecho E1 instanciada R1 instanciada	rebatible
	D2	`gallina(tina)`, `asustada(tina)`, `vuela(tina)`	hecho hecho R3 instanciada	rebatible
`~vuela(tina)`	D3	`gallina(tina)`, `~vuela(tina)`	hecho R2 instanciada	rebatible
`asustada(tina)`	D4	`asustada(tina)`	hecho	estricta
`vuela(peluche)`	D5	`gallina(peluche)`, `ave(peluche)`, `vuela(peluche)`	hecho E1 instanciada R1 instanciada	rebatible
`~vuela(peluche)`	D6	`~vuela(peluche)`	hecho	estricta
	D7	`gallina(peluche)`, `~vuela(peluche)`	hecho R2 instanciada	rebatible

- R3 no sirve para `vuela(peluche)`: no hay forma de derivar `asustada(peluche)`.
- R4 no aparece porque ninguno de los literales pedidos es `anida_en_ramas_altas(...)`.

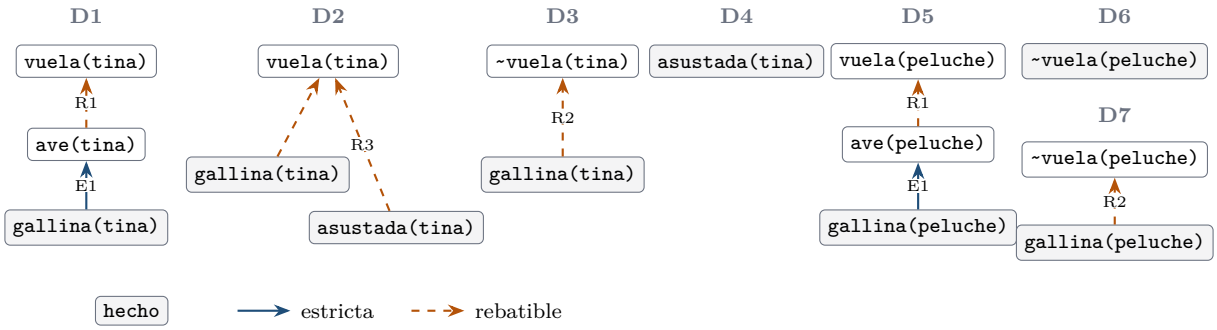


Figura 3: Las derivaciones D1–D7 como árboles: cada literal se obtiene de los de abajo con la regla indicada.

EXTRAS

- D1 y D5 muestran que una derivación puede usar **los dos tipos de reglas** (E1 estricta y R1 rebatible).[22]
- Los literales complementarios respecto de "~" (como `vuela(tina)` y `~vuela(tina)`) también se llaman **literales contradictorios** o **en conflicto**, porque representan dos conclusiones contradictorias.[23]

[22] Cátedra, Episodio X, parte 2, «Derivaciones para literales complementarios».
[23] Cátedra, Episodio X, parte 2, «Derivaciones para literales complementarios».

b)

Consigna

¿Cuándo se dice que un conjunto de reglas es contradictorio? Brinde un ejemplo.

Un conjunto S de hechos, reglas estrictas y rebatibles es **contradictorio** si a partir de S se pueden obtener derivaciones rebatibles tanto para un literal p como para su complemento $\sim p$.[24]

Ejemplo (de la cátedra):[25]

$$\Pi_c = \left\{ \begin{array}{l} \texttt{en_supermercado.} \\ \texttt{en_casa.} \\ \texttt{\~{}en_casa} \leftarrow \texttt{en_supermercado.} \end{array} \right\}$$

- `en_casa` se deriva porque es un hecho.
- `~en_casa` se deriva con la secuencia `en_supermercado`, `~en_casa`.

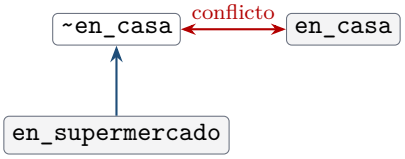


Figura 4: Π_c deriva dos literales complementarios: es contradictorio.

c)

Consigna

¿Cuál es la utilidad de que un programa lógico rebatible sea contradictorio? ¿Tiene sentido que el conjunto Π sea contradictorio?

Utilidad de un PLR contradictorio. Es lo que permite representar:[26]

- **información tentativa**: reglas generales y reglas para sus **excepciones**. En $\mathcal{P}_2$, R1 (las aves vuelan) tiene la excepción R2 (las gallinas no vuelan), que a su vez tiene la excepción R3 (las gallinas asustadas vuelan);
- **información potencialmente en conflicto**: distintas posturas u opiniones, que pueden venir de distintas fuentes.

Las contradicciones no se evitan: las resuelve el mecanismo de inferencia, analizando los argumentos a favor y en contra.

¿Π contradictorio? No. En todo PLR (Π, Δ) **se asume que Π no es contradictorio**: los hechos y las reglas estrictas (información segura) no pueden inferir conclusiones contradictorias.[27]

- Un conjunto contradictorio como Π_c del inciso b **no es válido** en DeLP como Π.
- Además, un argumento A exige que $\Pi \cup A$ no sea contradictorio. Con Π contradictorio, $\Pi \cup A$ lo sería para todo A, y no habría ningún argumento.[28]

[24] Cátedra, Episodio X, parte 2, «Conjunto de reglas contradictorio».
[25] Cátedra, Episodio X, parte 2, «Conjunto de reglas contradictorio».
[26] Cátedra, Episodio X, parte 2, «Defeasible Logic Programing (DeLP)».
[27] Cátedra, Episodio X, parte 2, «Conjunto de reglas contradictorio».
[28] Cátedra, Episodio X, parte 2, «Argumento».

d)

Consigna

Justifique por qué $(\Pi_2 \cup \Delta_2)$ es contradictorio, y por qué Π_2 no es contradictorio.

- $\Pi_2 \cup \Delta_2$ **es contradictorio**: deriva dos pares de literales complementarios (inciso a):
  - `vuela(tina)` (D1 o D2) y `~vuela(tina)` (D3);
  - `vuela(peluche)` (D5) y `~vuela(peluche)` (D6 o D7).
- Π_2 **no es contradictorio**: solo con hechos y E1 se derivan los cuatro hechos más `ave(tina)` y `ave(peluche)` (E1 aplicada a cada gallina). Entre estos seis literales no hay ningún par p, $\sim p$: el único literal negado, `~vuela(peluche)`, no tiene su complemento `vuela(peluche)`, que solo se obtiene con R1.

Literal	**Desde** Π_2	**Desde** $\Pi_2 \cup \Delta_2$
`asustada(tina)`, `gallina(tina)`, `gallina(peluche)`	sí (hechos)	sí
`ave(tina)`, `ave(peluche)`	sí (E1)	sí
`~vuela(peluche)`	sí (hecho)	sí
`vuela(peluche)`	no	**sí** (D5) ⇒ conflicto
`vuela(tina)`	no	**sí** (D1, D2)
`~vuela(tina)`	no	**sí** (D3) ⇒ conflicto con la anterior

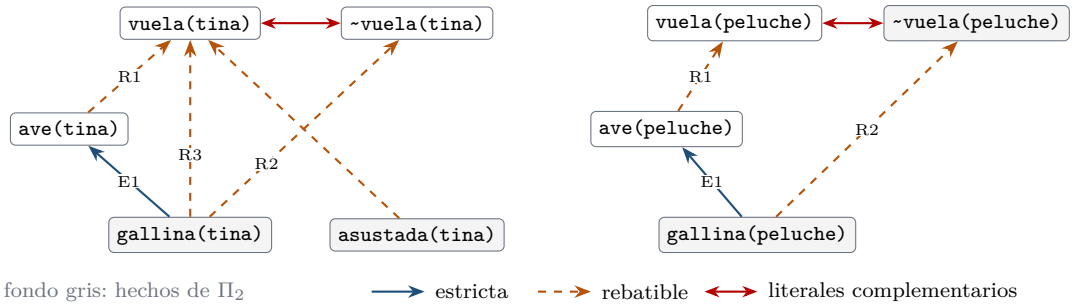


Figura 5: Lo que se deriva de $\Pi_2 \cup \Delta_2$ para `vuela`. Los dos conflictos aparecen recién al usar reglas rebatibles (líneas punteadas).

Ejercicio 3: Argumentos

a)

Consigna

¿Cómo se define formalmente la noción de *argumento* en DeLP? Explique cada una de las condiciones exigidas en la definición de argumento.

Un **argumento** para una conclusión h a partir de un PLR (Π, Δ) es un par $\langle A, h \rangle$, donde h es un literal fijo y A es un conjunto de **reglas rebatibles instanciadas** de Δ que cumple:[29]

1. **Derivación**: existe una derivación rebatible para h a partir de $A \cup \Pi$.
2. **Consistencia**: $A \cup \Pi$ no es contradictorio.
3. **Minimalidad**: no existe un subconjunto propio de A que cumpla (1) y (2).

[29] Cátedra, Episodio X, parte 2, «Argumento».

Qué pide cada condición. Con los ejemplos de la cátedra, a partir de:[30]

$$\Pi_n = \left\{ \begin{array}{l} \texttt{~en_casa} \leftarrow \texttt{en_supermercado.} \\ \texttt{en_supermercado.} \\ \texttt{de_noche.} \\ \texttt{festejo.} \end{array} \right\} \qquad \Delta_n = \left\{ \begin{array}{l} \texttt{en_casa} \prec \texttt{de_noche.} \\ \texttt{preparo_cena} \prec \texttt{de_noche.} \\ \texttt{~preparo_cena} \prec \texttt{festejo.} \\ \texttt{acuesto_tarde} \prec \texttt{festejo.} \\ \texttt{en_oficina} \prec \texttt{trabajando.} \end{array} \right\}$$

- **(1) Derivación**: las reglas de A, junto con los hechos y reglas estrictas, tienen que permitir **llegar a** h. $Q = \{\texttt{en_oficina} \prec \texttt{trabajando}\}$ no es argumento para `en_oficina`: `trabajando` no se deriva, así que $Q \cup \Pi_n$ no deriva `en_oficina`.
- **(2) Consistencia**: las reglas de A **no pueden contradecir** lo que se sabe con seguridad ni contradecirse entre sí. $R = \{\texttt{en_casa} \prec \texttt{de_noche}\}$ no es argumento para `en_casa`: $R \cup \Pi_n$ deriva `en_casa` y `~en_casa`.
- **(3) Minimalidad**: que **no haya reglas de más**; todas son necesarias para la derivación. $S = \{\texttt{~preparo_cena} \prec \texttt{festejo}, \texttt{acuesto_tarde} \prec \texttt{festejo}\}$ no es argumento para `~preparo_cena`, porque $B = \{\texttt{~preparo_cena} \prec \texttt{festejo}\} \subset S$ ya cumple (1) y (2). $\langle B, \texttt{~preparo_cena}\rangle$ sí es argumento.

Condición	**Exige**	**Contraejemplo en** (Π_n, Δ_n)	**Por qué falla**
(1) Derivación	$A \cup \Pi$ deriva h	Q para `en_oficina`	no deriva `trabajando`
(2) Consistencia	$A \cup \Pi$ no contradictorio	R para `en_casa`	deriva `en_casa` y `~en_casa`
(3) Minimalidad	ningún $A' \subset A$ cumple (1) y (2)	S para `~preparo_cena`	$B \subset S$ ya alcanza

EXTRAS

- **Observaciones** sobre la definición:[31]
  - el conjunto incluye **solo reglas rebatibles** (instanciadas); los hechos y reglas estrictas que se usan quedan en Π;
  - la conclusión es **única** y está instanciada;
  - se dice que $\langle A, h\rangle$ **sustenta** h;
  - para una misma conclusión puede haber **varios argumentos**, cada uno una razón diferente;
  - el argumento para un **hecho** es el **conjunto vacío**.
- **Notación gráfica**: un argumento se dibuja como un **triángulo**, con la conclusión en el vértice superior y el nombre del argumento adentro. Un sub-argumento es un triángulo dentro de otro.[32]

[30] Cátedra, Episodio X, parte 2, «Algunos detalles sobre las condiciones de los argumentos».
[31] Cátedra, Episodio X, parte 2, «Argumento».
[32] Cátedra, Episodio X (tercera parte), «Notación gráfica de argumentos».

b)

Consigna

Considerando el programa $\mathcal{P}_2$ del ejercicio anterior, muestre los argumentos posibles para cada uno de los siguientes literales: vuela(tina), ~vuela(tina), asustada(tina), anida_en_ramas_altas(tina).

Programa $\mathcal{P}_2$, con los nombres de reglas del ejercicio 2:

$$\Pi_2 = \left\{ \begin{array}{l} asustada(tina) \\ gallina(tina) \\ gallina(peluche) \\ \sim vuela(peluche) \\ ave(X) \leftarrow gallina(X) \end{array} \right\} \qquad \Delta_2 = \left\{ \begin{array}{l} vuela(X) \prec ave(X) \\ \sim vuela(X) \prec gallina(X) \\ vuela(X) \prec gallina(X), asustada(X) \\ anida_en_ramas_altas(X) \prec vuela(X) \end{array} \right\}$$

E1: `ave(X)` $\leftarrow$ `gallina(X)`
R1: `vuela(X)` $\prec$ `ave(X)`
R2: `~vuela(X)` $\prec$ `gallina(X)`
R3: `vuela(X)` $\prec$ `gallina(X), asustada(X)`
R4: `anida_en_ramas_altas(X)` $\prec$ `vuela(X)`

Conclusión	**Argumento**	**Derivación**
`vuela(tina)`	A_1 = {`vuela(tina)` $\prec$ `ave(tina)`}	D1
	A_2 = {`vuela(tina)` $\prec$ `gallina(tina), asustada(tina)`}	D2
`~vuela(tina)`	B = {`~vuela(tina)` $\prec$ `gallina(tina)`}	D3
`asustada(tina)`	$\emptyset$ (es un hecho)	D4
`anida_en_ramas_altas(tina)`	C_1 = {`anida_en_ramas_altas(tina)` $\prec$ `vuela(tina)`, `vuela(tina)` $\prec$ `ave(tina)`}	D1 + R4
	C_2 = {`anida_en_ramas_altas(tina)` $\prec$ `vuela(tina)`, `vuela(tina)` $\prec$ `gallina(tina), asustada(tina)`}	D2 + R4

Por qué cumplen las tres condiciones:

- **(1)**: cada uno contiene exactamente las reglas rebatibles de la derivación indicada (D1–D4 del ejercicio 2; para C_1 y C_2, esa derivación seguida de `anida_en_ramas_altas(tina)` por R4).
- **(2)**: con Π_2, ninguno deriva un literal y su complemento. Con Π_2 solo se deriva un literal negado, `~vuela(peluche)`, y estos argumentos no derivan `vuela(peluche)`. Además, ninguno junta R2 con R1 o R3 para tina.
- **(3)**: sacando cualquier regla ya no se deriva la conclusión.

Conjuntos que **no** son argumentos (ejemplos):

- $A_1 \cup A_2$ para `vuela(tina)`: falla (3), porque $A_1 \subset A_1 \cup A_2$ ya alcanza.
- $C_1 \cup B$ para `anida_en_ramas_altas(tina)`: falla (2), porque deriva `vuela(tina)` y `~vuela(tina)` (y también (3)).

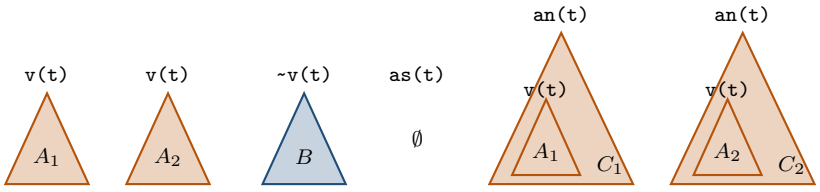


Figura 6: Los argumentos de b en notación de triángulos (`v(t)` = `vuela(tina)`, `as(t)` = `asustada(tina)`, `an(t)` = `anida_en_ramas_altas(tina)`). A_1 y A_2 son sub-argumentos de C_1 y C_2.

EXTRAS

- Un argumento $\langle S, q\rangle$ es **sub-argumento** de $\langle X, h\rangle$ si $S \subseteq X$. Por eso $\langle A_1, \texttt{vuela(tina)}\rangle$ es sub-argumento de $\langle C_1, \texttt{anida_en_ramas_altas(tina)}\rangle$ (y A_2 de C_2).[33]

---

c)

Consigna

¿Qué relación existe entre las nociones de argumento y derivación?

- Todo argumento **necesita una derivación**: es la condición (1), una derivación rebatible de h a partir de $A \cup \Pi$.[34]
- El conjunto A son las **reglas rebatibles instanciadas que usa esa derivación**. En el ejemplo de la cátedra, N tiene las dos reglas rebatibles (R2 y R5) de la derivación de `~comprar_acciones(acme)`; la regla estricta E3 y el hecho que usa la derivación quedan en Π.[35]
- La recíproca no vale: **no toda derivación da un argumento**. Además de derivar, A tiene que ser consistente con Π (2) y mínimo (3). Ejemplo: `vuela(peluche)` (inciso d).
- Si la derivación es **estricta**, no usa reglas rebatibles y el argumento es $\emptyset$ (p. ej. D4 para `asustada(tina)`).[36]

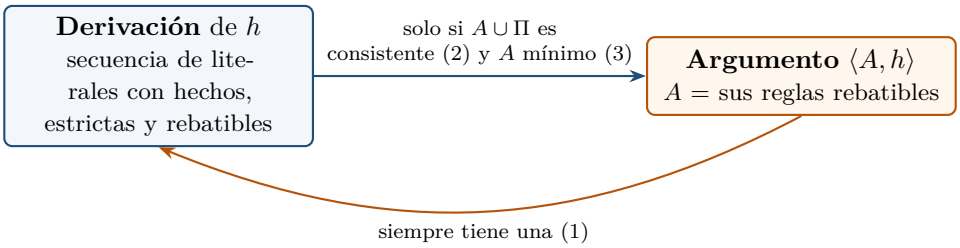


Figura 7: Todo argumento se apoya en una derivación; no toda derivación da un argumento.

d)

Consigna

¿Por qué a partir del programa $\mathcal{P}_2$ existe una derivación para vuela(peluche) pero no existe un argumento para vuela(peluche)?

- **Derivación**: D5 = `gallina(peluche)`, `ave(peluche)` (E1), `vuela(peluche)` (R1). Una derivación solo pide que cada literal se obtenga de un hecho o una regla; no controla contradicciones.
- **Argumento**: el único candidato es $A = \{\texttt{vuela(peluche)} \prec \texttt{ave(peluche)}\}$ (R3 no sirve: `asustada(peluche)` no se deriva). Pero `~vuela(peluche)` es un **hecho** de Π_2, así que $A \cup \Pi_2$ deriva `vuela(peluche)` y `~vuela(peluche)`: es contradictorio y **no cumple la condición (2)**.
- Agregar reglas no lo arregla: cualquier conjunto que derive `vuela(peluche)`, unido con Π_2, sigue derivando también el hecho `~vuela(peluche)`.

[33] Cátedra, Episodio X (tercera parte), «Relaciones entre argumentos: sub-argumento».
[34] Cátedra, Episodio X, parte 2, «Argumento».
[35] Cátedra, Episodio X, parte 2, «Argumento».
[36] Cátedra, Episodio X, parte 2, «Argumento».

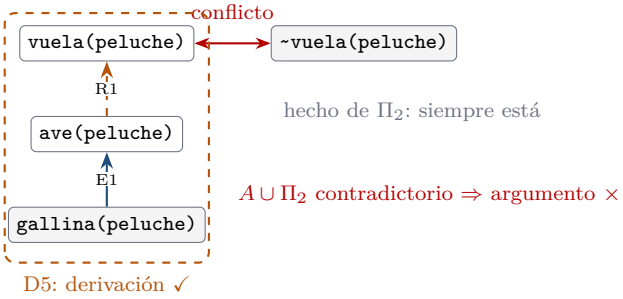


Figura 8: `vuela(peluche)` tiene derivación, pero choca con el hecho `~vuela(peluche)`.

Ejercicio 4: Ataques y derrotas

a)

Consigna

Defina formalmente las nociones de contra-argumento y derrotador (distinguiendo entre derrotador propio y derrotador por bloqueo).

Definiciones, dado un PLR (Π, Δ):

- **Desacuerdo**: $\langle X, f\rangle$ y $\langle Y, g\rangle$ están en desacuerdo (o en conflicto) si $\Pi \cup \{f, g\}$ es contradictorio.[37]
- **Contra-argumento (ataque)**: $\langle X, p\rangle$ ataca (contra-argumenta) a $\langle Y, h\rangle$ si existe un sub-argumento $\langle S, q\rangle$ de $\langle Y, h\rangle$ tal que $\langle X, p\rangle$ y $\langle S, q\rangle$ están en desacuerdo.[38]
  - $\langle S, q\rangle$ se llama **sub-argumento de desacuerdo**; puede ser el propio $\langle Y, h\rangle$.
  - q es el **punto de ataque**.
- **Derrotador**: $\langle X, p\rangle$ derrota a $\langle Y, h\rangle$ si lo ataca con sub-argumento de desacuerdo $\langle S, q\rangle$ y se cumple una de estas condiciones:[39]

  (a) $\langle X, p\rangle$ es **preferido** a $\langle S, q\rangle$: es un **derrotador propio**;

  (b) $\langle X, p\rangle$ **no es comparable** con $\langle S, q\rangle$: es un **derrotador por bloqueo**.

  La derrota combina ataque y preferencia, y la preferencia se evalúa entre el atacante y el **sub-argumento de desacuerdo** (no el argumento atacado completo).

$\langle X, p\rangle$ **ataca a** $\langle Y, h\rangle$ **en** $\langle S, q\rangle$**, y...**	**¿Derrota?**	**Tipo**
$\langle X, p\rangle$ es preferido a $\langle S, q\rangle$	sí	derrotador propio
$\langle X, p\rangle$ y $\langle S, q\rangle$ no son comparables	sí	derrotador por bloqueo
$\langle S, q\rangle$ es preferido a $\langle X, p\rangle$	no	(solo ataque)

[37] Cátedra, Episodio X (tercera parte), «Relaciones entre argumentos: desacuerdo».
[38] Cátedra, Episodio X (tercera parte), «Ataque (contra-argumento)».
[39] Cátedra, Episodio X (tercera parte), «Derrota propia y derrota por bloqueo».

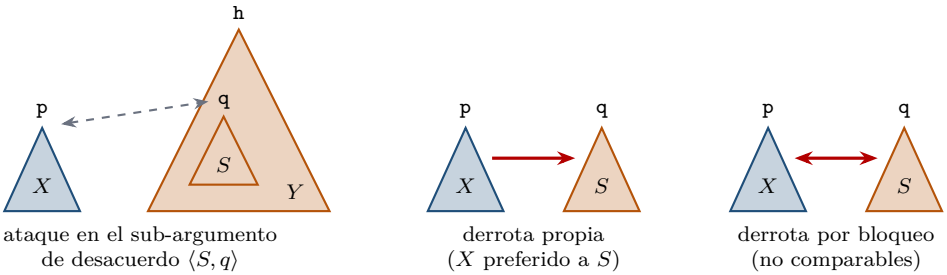


Figura 9: Ataque (flecha gris punteada), derrota propia (flecha simple) y derrota por bloqueo (flecha doble), como en la notación de la cátedra.

EXTRAS

- Todo ataque a un sub-argumento es también un ataque al súper-argumento, y si R derrota a un sub-argumento X de N, también derrota a N.[40]
- Si R derrota a N, R **defiende indirectamente** todo lo que N ataca; y un D que ataque a R defiende indirectamente a N (y ataca indirectamente lo que N ataca).[41]

b)

Consigna

A partir de los argumentos formados en el ejercicio 3.b, indique los ataques que aparecen entre los argumentos para vuela(tina) y los argumentos para ~vuela(tina).

Argumentos del ejercicio 3.b (nombres de reglas del ejercicio 2):

- $\langle A_1, \texttt{vuela(tina)}\rangle$, con $A_1 = \{\texttt{vuela(tina)} \prec \texttt{ave(tina)}\}$ (R1);
- $\langle A_2, \texttt{vuela(tina)}\rangle$, con $A_2 = \{\texttt{vuela(tina)} \prec \texttt{gallina(tina), asustada(tina)}\}$ (R3);
- $\langle B, \texttt{~vuela(tina)}\rangle$, con $B = \{\texttt{~vuela(tina)} \prec \texttt{gallina(tina)}\}$ (R2).

Ataques:

- A_1 y B **se atacan mutuamente**: $\Pi_2 \cup \{\texttt{vuela(tina)}, \texttt{~vuela(tina)}\}$ es contradictorio.
- A_2 y B **se atacan mutuamente**, por la misma razón.
- Cada argumento tiene una sola regla, así que su único sub-argumento es él mismo: el sub-argumento de desacuerdo es el **argumento completo** y el punto de ataque es la **conclusión**.
- A_1 y A_2 **no se atacan**: tienen la misma conclusión, no están en desacuerdo.

Atacante	Atacado	Sub-argumento de desacuerdo	Punto de ataque
$\langle A_1, \texttt{vuela(tina)}\rangle$	$\langle B, \texttt{~vuela(tina)}\rangle$	$\langle B, \texttt{~vuela(tina)}\rangle$	~vuela(tina)
$\langle B, \texttt{~vuela(tina)}\rangle$	$\langle A_1, \texttt{vuela(tina)}\rangle$	$\langle A_1, \texttt{vuela(tina)}\rangle$	vuela(tina)
$\langle A_2, \texttt{vuela(tina)}\rangle$	$\langle B, \texttt{~vuela(tina)}\rangle$	$\langle B, \texttt{~vuela(tina)}\rangle$	~vuela(tina)
$\langle B, \texttt{~vuela(tina)}\rangle$	$\langle A_2, \texttt{vuela(tina)}\rangle$	$\langle A_2, \texttt{vuela(tina)}\rangle$	vuela(tina)

[40] Cátedra, Episodio X (tercera parte), «Relaciones entre argumentos».
[41] Cátedra, Episodio X (tercera parte), «Relaciones entre argumentos».

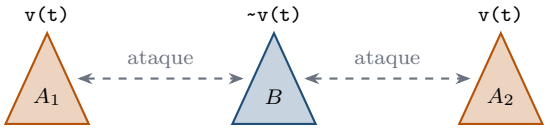


Figura 10: Ataques entre los argumentos para `vuela(tina)` (`v(t)`) y `~vuela(tina)`: B ataca y es atacado por A_1 y por A_2.

EXTRAS

- B también ataca a $\langle C_1, \texttt{anida_en_ramas_altas(tina)}\rangle$ y a $\langle C_2, \texttt{anida_en_ramas_altas(tina)}\rangle$ del ejercicio 3.b, con sub-argumentos de desacuerdo A_1 y A_2 (punto de ataque `vuela(tina)`): todo ataque a un sub-argumento es un ataque al súper-argumento. En cambio, C_1 y C_2 no atacan a B: $\Pi_2 \cup \{\texttt{anida_en_ramas_altas(tina)}, \texttt{~vuela(tina)}\}$ no es contradictorio.[42]

c)

Consigna

Defina un orden de preferencia para el conjunto de reglas rebatibles Δ_2 y, a partir de las relaciones de ataque presentadas en el inciso anterior, indique qué derrotas se producen. Para cada una, indique si se trata de una derrota propia o por bloqueo.

Orden elegido. Se prioriza la regla más específica (la excepción sobre la regla general):

R2 > R1 (las gallinas no vuelan, aunque sean aves) R3 > R2 (las gallinas asustadas vuelan)

Por transitividad, también R3 > R1. R4 queda sin relacionar (no aparece en los argumentos que se comparan).

Criterio de preferencia (prioridad entre reglas). N es preferido a A si:[43]

1. existe $R_1 \in N$ y $R_2 \in A$ con $R_1 > R_2$ (N tiene al menos una regla mejor que una de A), y
2. no existe $R_1 \in N$ y $R_2 \in A$ con $R_1 < R_2$ (A no tiene ninguna regla mejor que una de N).

Comparaciones.

- B vs. A_1: R2 > R1 y A_1 no tiene reglas mejores $\Rightarrow$ B **es preferido a** A_1.
- A_2 vs. B: R3 > R2 y B no tiene reglas mejores $\Rightarrow$ A_2 **es preferido a** B.

Derrotas.

Ataque	Preferencia	¿Derrota?	Tipo
B ataca a A_1	B preferido a A_1	**sí**	**propia**
A_1 ataca a B	B preferido a A_1	no	—
A_2 ataca a B	A_2 preferido a B	**sí**	**propia**
B ataca a A_2	A_2 preferido a B	no	—

- $\langle B, \texttt{~vuela(tina)}\rangle$ es **derrotador propio** de $\langle A_1, \texttt{vuela(tina)}\rangle$.

[42] Cátedra, Episodio X (tercera parte), «Relaciones entre argumentos».
[43] Cátedra, Episodio X (tercera parte), «Preferencia entre argumentos».

- $\langle A_2, \texttt{vuela(tina)} \rangle$ es **derrotador propio** de $\langle B, \texttt{\sim vuela(tina)} \rangle$.
- Con este orden **no hay derrotas por bloqueo**: todos los pares que se atacan son comparables.

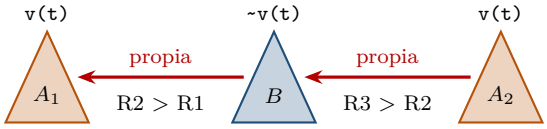
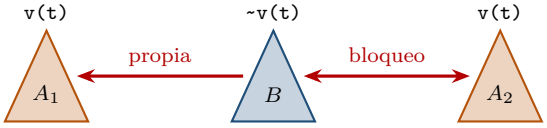


Figura 11: Derrotas con el orden R3 > R2 > R1: B derrota a A_1 y A_2 derrota a B, ambas derrotas propias.

EXTRAS

- **Otro orden posible**: solo R2 > R1 (sin comparar R3 con R2). Entonces A_2 y B no son comparables y **se derrotan mutuamente por bloqueo**; B sigue siendo derrotador propio de A_1. Es el mismo caso que $\langle A, g \rangle$ y $\langle C, \sim g \rangle$ en el ejemplo de la cátedra.[44]



- Con el orden elegido, A_2 derrota a B, que derrota a A_1: A_2 **defiende indirectamente** a A_1.[45]

Ejercicio 5

Consigna

Considere el programa $\mathcal{P}_5 = (\Pi_5, \Delta_5)$:

$$\Pi_5 = \left\{ \begin{array}{l} \mathit{viento} \leftarrow \mathit{viento_caluroso} \\ \mathit{calor} \\ \mathit{caluroso} \\ \mathit{viento_caluroso} \\ \mathit{ahorrar_energia} \end{array} \right\} \qquad \Delta_5 = \left\{ \begin{array}{l} \mathit{aire_acond} \prec \mathit{calor} \\ \mathit{aire_acond} \prec \mathit{sofocante} \\ \sim \mathit{aire_acond} \prec \mathit{abrir_ventana} \\ \sim \mathit{aire_acond} \prec \mathit{ahorrar_energia} \\ \mathit{abrir_ventana} \prec \mathit{calor}, \mathit{viento} \\ \sim \mathit{abrir_ventana} \prec \mathit{viento_caluroso} \end{array} \right\}$$

En lo que sigue, las reglas se nombran así:

R1:	`aire_acond` $\prec$ `calor`	R4:	`~aire_acond` $\prec$ `ahorrar_energia`
R2:	`aire_acond` $\prec$ `sofocante`	R5:	`abrir_ventana` $\prec$ `calor, viento`
R3:	`~aire_acond` $\prec$ `abrir_ventana`	R6:	`~abrir_ventana` $\prec$ `viento_caluroso`

`sofocante` no se deriva

R2 nunca se puede usar: `sofocante` no es un hecho ni cabeza de ninguna regla. En Π_5 está el hecho `caluroso`, que ninguna regla usa; probablemente R2 debía ser `aire_acond` $\prec$ `caluroso`. Se resuelve el programa tal como está.

[44] Cátedra, Episodio X (tercera parte), «Derrota propia y derrota por bloqueo».
[45] Cátedra, Episodio X (tercera parte), «Relaciones entre argumentos».

a)

Consigna

Brinde dos argumentos para no encender el aire acondicionado (~aire_acond) y muestre todos los contra-argumentos que los atacan.

Argumentos para `~aire_acond`.

- $\langle A_1, \texttt{~aire_acond}\rangle$, con $A_1 = \{\texttt{~aire_acond} \prec \texttt{abrir_ventana}, \texttt{abrir_ventana} \prec \texttt{calor, viento}\}$ (R3, R5). `calor` es hecho y `viento` sale de la regla estricta con `viento_caluroso`.
  - Tiene el sub-argumento $\langle D, \texttt{abrir_ventana}\rangle$, con $D = \{\texttt{abrir_ventana} \prec \texttt{calor, viento}\}$ (R5).
- $\langle A_2, \texttt{~aire_acond}\rangle$, con $A_2 = \{\texttt{~aire_acond} \prec \texttt{ahorrar_energia}\}$ (R4).

Contra-argumentos. Un contra-argumento tiene que estar en desacuerdo con algún sub-argumento: concluir `aire_acond` (contra A_1 o A_2) o `~abrir_ventana` (contra D).[46]

- $\langle B, \texttt{aire_acond}\rangle$, con $B = \{\texttt{aire_acond} \prec \texttt{calor}\}$ (R1). Es el único argumento para `aire_acond` (R2 no se puede usar).
- $\langle C, \texttt{~abrir_ventana}\rangle$, con $C = \{\texttt{~abrir_ventana} \prec \texttt{viento_caluroso}\}$ (R6).

Contra-argumento	Ataca a	Sub-argumento de desacuerdo	Punto de ataque
$\langle B, \texttt{aire_acond}\rangle$	$\langle A_1, \texttt{~aire_acond}\rangle$	$\langle A_1, \texttt{~aire_acond}\rangle$	`~aire_acond`
$\langle C, \texttt{~abrir_ventana}\rangle$	$\langle A_1, \texttt{~aire_acond}\rangle$	$\langle D, \texttt{abrir_ventana}\rangle$	`abrir_ventana`
$\langle B, \texttt{aire_acond}\rangle$	$\langle A_2, \texttt{~aire_acond}\rangle$	$\langle A_2, \texttt{~aire_acond}\rangle$	`~aire_acond`

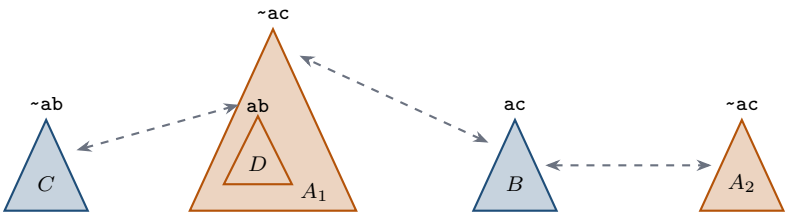


Figura 12: Ataques a los argumentos para `~aire_acond` (`ac` = `aire_acond`, `ab` = `abrir_ventana`). C ataca a A_1 en su sub-argumento D.

b)

Consigna

Compare los argumentos obtenidos a partir del criterio de preferencia entre reglas visto en clase. En este caso, considere el siguiente orden:

$$(\texttt{aire_acond} \prec \texttt{sofocante}) > (\texttt{~aire_acond} \prec \texttt{ahorrar_energia})$$
$$(\texttt{~aire_acond} \prec \texttt{ahorrar_energia}) > (\texttt{aire_acond} \prec \texttt{calor})$$
$$(\texttt{~abrir_ventana} \prec \texttt{viento_caluroso}) > (\texttt{abrir_ventana} \prec \texttt{calor, viento})$$

Determine si los contra-argumentos obtenidos en el inciso anterior son derrotadores. En caso afirmativo, indique si son derrotadores propios o por bloqueo.

Con los nombres de reglas: **R2** $>$ **R4**, **R4** $>$ **R1**, **R6** $>$ **R5** (y R2 $>$ R1 por transitividad). R3 no está relacionada con ninguna regla.

[46] Cátedra, Episodio X (tercera parte), «Ataque (contra-argumento)».

La preferencia se evalúa entre el atacante y el **sub-argumento de desacuerdo**; N es preferido a A si tiene una regla mejor que una de A y A no tiene ninguna mejor que una de N.[47]

Atacante	Sub-arg. de desacuerdo	Reglas	Comparación	Resultado
B	A_1	{R1} vs. {R3, R5}	sin relación entre R1 y R3, R5	**bloqueo**
C	D (de A_1)	{R6} vs. {R5}	R6 > R5: C preferido	**propio**
B	A_2	{R1} vs. {R4}	R4 > R1: A_2 preferido	no derrota

- $\langle B, \texttt{aire_acond} \rangle$ es **derrotador por bloqueo** de $\langle A_1, \texttt{\~aire_acond} \rangle$: no son comparables.
- $\langle C, \texttt{\~abrir_ventana} \rangle$ es **derrotador propio** de $\langle A_1, \texttt{\~aire_acond} \rangle$: es preferido a D, el sub-argumento atacado.
- $\langle B, \texttt{aire_acond} \rangle$ ataca a $\langle A_2, \texttt{\~aire_acond} \rangle$ pero **no es derrotador**: A_2 es preferido a B.
- La regla R2 aparece en el orden pero no influye: no está en ningún argumento.

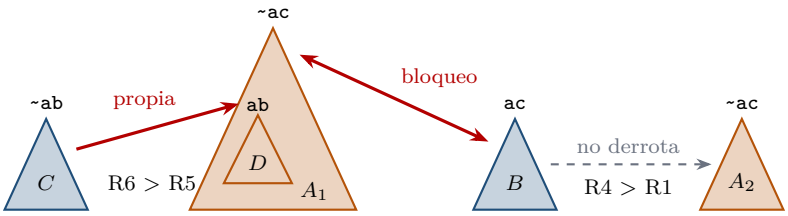


Figura 13: Derrotas sobre los argumentos para `~aire_acond`: C derrota a A_1 (propia, en D), B y A_1 se bloquean, y B no derrota a A_2.

EXTRAS

- **Ataques en sentido contrario**, que se usan en los ejercicios 6 y 8:[48]
  - A_1 ataca a B y, como no son comparables, también es su **derrotador por bloqueo**.
  - A_2 ataca a B y es su **derrotador propio** (R4 > R1).
  - D ataca a C pero no lo derrota (C es preferido). A_1 no ataca a C: `~aire_acond` y `~abrir_ventana` junto con Π_5 no forman un conjunto contradictorio; el ataque es solo de su sub-argumento D.

[47] Cátedra, Episodio X (tercera parte), «Derrota propia y derrota por bloqueo»; «Preferencia entre argumentos».
[48] Cátedra, Episodio X (tercera parte), «Derrota propia y derrota por bloqueo».